浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：April 24, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　　指导老师：魏准　　实验类型：设计型实验
实验名称：Delta 规则神经网络训练　　成　绩：　　签　名：

装 订 线

一、 Delta 规则神经网络训练——实验 5-1

1. 实验目的

构建以 Sigmoid 为激活函数的单层感知器神经网络，分别采用 **SGD**（随机梯度下降，Stochastic Gradient Descent）和 **Batch**（批量梯度下降）两种训练方法，结合 Delta 规则对网络权重进行迭代更新，学习映射关系 $d = x_1$（输出等于第一个输入特征）。具体任务包括：

- 通过 SGD 训练方法及 Delta 规则对神经网络进行训练，输出训练后的结果；
- 通过 Batch 训练方法及 Delta 规则对神经网络进行训练，输出训练后的结果；
- 比较两种训练方法的误差（MSE）随 epoch 变化趋势，并可视化结果；
- 讨论不同学习率（alpha）对训练结果的影响；
- 讨论不同初始化权重 W 取值范围对训练结果的影响。

2. 实验原理

网络结构与训练数据　本实验采用单层感知器结构：3 个输入（x_1、x_2、x_3，其中 $x_3 \equiv 1$ 可视作为偏置项），1 个 Sigmoid 输出神经元，权重矩阵 $W \in \mathbb{R}^{1\times 3}$。训练数据集如表 1 所示，共 4 条样本，目标是学习 $d = x_1$（输出等于第一个特征，与 x_2 无关）：

x_1	x_2	x_3 (偏置)	d
0	0	1	0
0	1	1	0
1	0	1	1
1	1	1	1

Table 1: 训练数据集（$d = x_1$）

Sigmoid 激活函数

$$\sigma(v) = \frac{1}{1+e^{-v}}, \quad \sigma'(v) = \sigma(v)\big(1-\sigma(v)\big)$$

均方误差（MSE） 训练过程中用均方误差衡量网络输出与期望输出之间的差距：

$$\mathrm{MSE} = \frac{1}{N}\sum_{k=1}^{N}\big(d^{(k)} - y^{(k)}\big)^2$$

其中 $N = 4$ 为样本数，$d^{(k)}$ 为期望输出，$y^{(k)}$ 为网络实际输出。

Delta 规则　对第 k 个样本，前向传播为：

$$v = W \cdot x^{(k)}, \quad y = \sigma(v), \quad e = d^{(k)} - y$$

权重更新量：

$$\Delta W_j = \alpha \cdot e \cdot y(1-y) \cdot x_j^{(k)}$$

SGD 与 Batch 的区别

- **SGD（随机梯度下降）**：每个 epoch 先随机打乱样本顺序，然后逐样本计算误差并立即更新权重，每个 epoch 共更新 $N = 4$ 次。
- **Batch（批量梯度下降）**：遍历全部样本后累加权重增量并取平均，再统一更新一次，每个 epoch 只更新 1 次。更新公式为：

$$\Delta W_j = \frac{1}{N} \sum_{k=1}^{N} \alpha \cdot e^{(k)} \cdot y^{(k)}(1-y^{(k)}) \cdot x_j^{(k)}$$

3. 核心代码实现

```python
# Sigmoid.py
import numpy as np

def Sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))
```

Listing 1: Step 1——Sigmoid 激活函数定义

```python
# DeltaSGD.py
import numpy as np
from Sigmoid import Sigmoid

def DeltaSGD(W, X, D, alpha=0.1):
    N = 4  # 训练数据个数
    idx = np.random.permutation(N)  # 每 epoch 随机打乱样本顺序
    for k in idx:
        x = X[k, :].T
        d = D[k]
        v = (W @ x)[0]
        y = Sigmoid(v)
        e = d - y
        W[0][0] = W[0][0] + alpha * e * y * (1 - y) * x[0]
        W[0][1] = W[0][1] + alpha * e * y * (1 - y) * x[1]
        W[0][2] = W[0][2] + alpha * e * y * (1 - y) * x[2]
    return W
```

Listing 2: Step 2——SGD 训练函数（每个样本后立即更新，每 epoch 随机打乱顺序）

```python
# DeltaBatch.py
import numpy as np
from Sigmoid import Sigmoid

def DeltaBatch(W, X, D, alpha=0.1):
    N = 4  # 训练数据个数
    dW = np.zeros_like(W)
    for k in range(N):
        x = X[k, :].T
        d = D[k]
        v = (W @ x)[0]
        y = Sigmoid(v)
        e = d - y
        dW[0][0] += alpha * e * y * (1 - y) * x[0]
        dW[0][1] += alpha * e * y * (1 - y) * x[1]
        dW[0][2] += alpha * e * y * (1 - y) * x[2]
    W = W + dW / N  # 除以 N 取平均，与 SGD 的 alpha 公平可比
    return W
```

Listing 3: Step 3——Batch 训练函数（遍历全部样本后取平均并统一更新）

装 订 线

装 订 线

```python
# exp5_1.py (初始化与训练, Step 3-4)
import numpy as np
from Sigmoid import Sigmoid
from DeltaSGD import DeltaSGD
from DeltaBatch import DeltaBatch

# 数据集（幻灯片 P25）：d = x1, x3=1 为偏置
X = np.array([[0, 0, 1],
              [0, 1, 1],
              [1, 0, 1],
              [1, 1, 1]], dtype=float)
D = np.array([0, 0, 1, 1], dtype=float)

np.random.seed(42)
W_init = np.random.uniform(-0.5, 0.5, (1, 3))
W_sgd   = W_init.copy()
W_batch = W_init.copy()

# SGD 训练 10000 epochs
for _epoch in range(10000):
    W_sgd = DeltaSGD(W_sgd, X, D)

# Batch 训练 10000 epochs
np.random.seed(42)
W_batch = W_init.copy()
for _epoch in range(10000):
    W_batch = DeltaBatch(W_batch, X, D)

# 输出预测值
N = 4
for k in range(N):
    x = X[k, :].T
    y = Sigmoid((W_sgd @ x)[0])
    print(f"SGD  x1={int(X[k,0])},x2={int(X[k,1])},x3={int(X[k,2])}, d={int(D[k])}, y={y:.4f}")
for k in range(N):
    x = X[k, :].T
    y = Sigmoid((W_batch @ x)[0])
    print(f"Batch x1={int(X[k,0])},x2={int(X[k,1])},x3={int(X[k,2])}, d={int(D[k])}, y={y:.4f}")
```

Listing 4: Step 4——数据集初始化、训练并输出预测结果

4. 运行结果

(1) SGD 训练结果

```
SGD 训练结果 (alpha=0.1, W in [-0.5, 0.5])
最终权重 W = [ 7.25748598 -0.22006916 -3.41199044]
  输入 x1=0, x2=0, x3=1, 期望: 0, 输出: 0.0319
  输入 x1=0, x2=1, x3=1, 期望: 0, 输出: 0.0258
  输入 x1=1, x2=0, x3=1, 期望: 1, 输出: 0.9791
  输入 x1=1, x2=1, x3=1, 期望: 1, 输出: 0.9741
```

训练后 $w_1 = 7.26$（较大正值，正确响应 x_1），$w_2 = -0.22$（接近零，说明网络正确地忽略了无关输入 x_2），$w_3 = -3.41$（负偏置，设置激活阈值）。当 $x_1 = 0$ 时输出 ≈ 0.03，当 $x_1 = 1$ 时输出 ≈ 0.97，成功学习了 $d = x_1$ 的映射。

(2) Batch 训练结果

```
Batch 训练结果 (alpha=0.1, W in [-0.5, 0.5])
最终权重 W = [ 5.69466547 -0.23504375 -2.61711933]
  输入 x1=0, x2=0, x3=1, 期望: 0, 输出: 0.0680
  输入 x1=0, x2=1, x3=1, 期望: 0, 输出: 0.0546
  输入 x1=1, x2=0, x3=1, 期望: 1, 输出: 0.9560
  输入 x1=1, x2=1, x3=1, 期望: 1, 输出: 0.9449
```

Batch 方法同样成功学习了 $d = x_1$ 的映射，但在相同 epoch 数和学习率下，最终权重绝对值更小（$w_1 = 5.69$ vs 7.26），预测值离 0/1 边界更远，精度略低于 SGD（输出范围约 0.05–0.97 vs 0.03–0.98）。

(3) SGD 与 Batch 的 MSE 比较

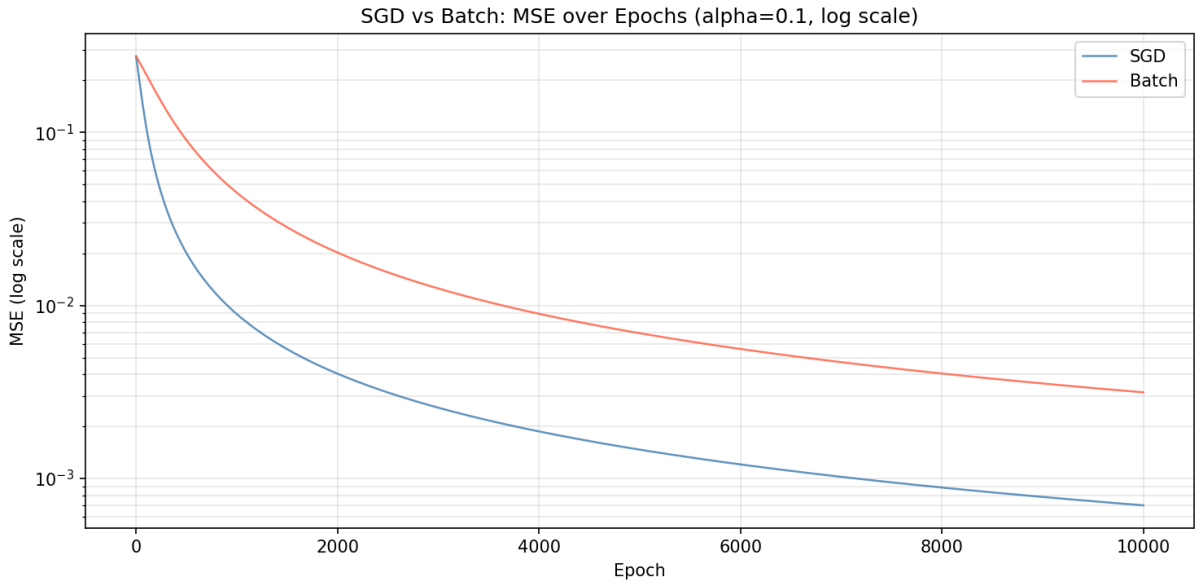


Figure 1: SGD 与 Batch 训练方法的 MSE 随 epoch 变化曲线（对数坐标，$\alpha = 0.1$，初始权重范围 $[-0.5, 0.5]$）

从图 1（纵轴为对数坐标，便于观察跨越两个数量级的收敛全程）可见：SGD（蓝色）收敛速度明显快于 Batch（红色），且 10000 轮后最终 MSE 也更低（SGD: 0.000699，Batch: 0.003145，相差约 4.5 倍）。在对数坐标下，SGD 曲线后期呈现出随机打乱样本顺序所带来的轻微锯齿状波动，而 Batch 曲线因每 epoch 对全样本取平均梯度，始终保持光滑单调下降，直观体现了两种方法在更新噪声上的本质差异。MSE 差距的根本原因在于：SGD 每个 epoch 执行 $N = 4$ 次权重更新，Batch 每个 epoch 仅执行 1 次，同等 epoch 数下 SGD 实际完成了 4 倍的更新量。

(4) 各样本输出值 y 随 epoch 的收敛过程

除聚合 MSE 外，图 2 展示了 4 个样本的网络输出 $y^{(k)}$ 在训练过程中逐 epoch 的变化轨迹，直观呈现” 真实结果与输出之差” 如何随训练收敛：

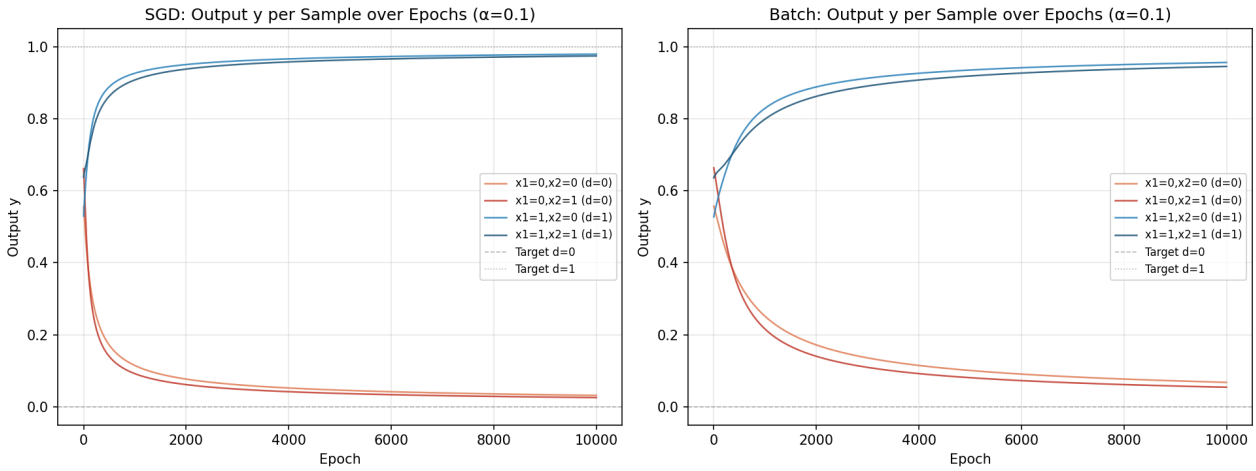


Figure 2: SGD（左）与 Batch（右）训练过程中各样本输出 y 随 epoch 变化轨迹。红色系为目标 $d=0$ 的两个样本（y 应向 0 收敛），蓝色系为目标 $d=1$ 的两个样本（y 应向 1 收敛）；灰色虚线为目标值 $d=0$ 和 $d=1$。

从图 2 可以看出：

- **收敛方向正确**：$d=0$ 的样本（红色，$x_1=0$）的输出从初始值约 0.5 出发逐步下降并趋向 0；$d=1$ 的样本（蓝色，$x_1=1$）的输出逐步上升并趋向 1；
- **SGD 收敛更快且更彻底**：SGD（左图）在约 1000–2000 epoch 内各曲线已基本到达目标附近；Batch（右图）收敛过程更平缓，10000 epoch 结束时离目标仍稍远（$y\approx 0.055$–0.068 vs 0，$y\approx 0.945$–0.976 vs 1），与 MSE 曲线结论一致；
- **同类样本几乎重叠**：$d=0$ 的两条曲线（$x_2=0$ 与 $x_2=1$）、$d=1$ 的两条曲线几乎重叠，说明网络正确学习了 $d=x_1$、与 x_2 无关的映射；
- **SGD 曲线有轻微波动**：在对数坐标 MSE 图中已可见的随机性，在输出轨迹图中也体现为蓝色/红色曲线的微小锯齿。

5. 不同学习率（alpha）对结果的影响

固定初始权重（$W\sim U(-0.5,0.5)$，`seed=42`），在 $\alpha\in\{0.01,\ 0.1,\ 0.5,\ 1.0,\ 5.0,\ 10.0,\ 50.0\}$ 下各训练 10000 epochs，MSE 收敛曲线如图 3 所示，最终 MSE 汇总如下：

```
不同学习率的最终 MSE (10000 epochs)
 Alpha |    SGD MSE |  Batch MSE
--------------------------------------
  0.01 |   0.008952 |   0.044961
   0.1 |   0.000699 |   0.003145
   0.5 |   0.000131 |   0.000552
   1.0 |   0.000065 |   0.000268
   5.0 |   0.000013 |   0.000052
  10.0 |   0.000006 |   0.000026
  50.0 |   0.000001 |   0.000005
```

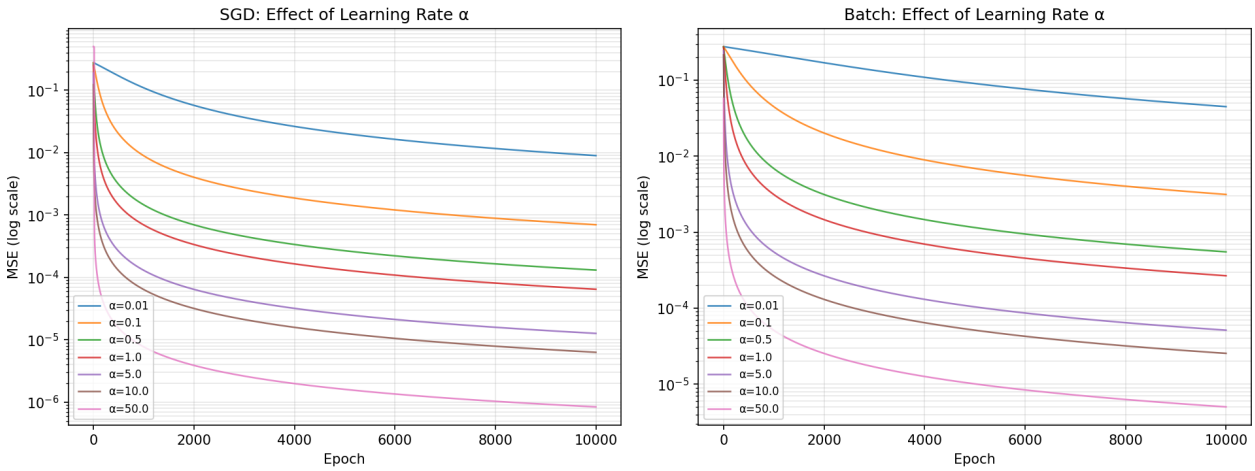


Figure 3: $\alpha \in \{0.01,\ 0.1,\ 0.5,\ 1.0,\ 5.0,\ 10.0,\ 50.0\}$ 下 SGD（左）与 Batch（右）的 MSE 随 epoch 变化曲线（对数坐标，10000 epochs）

分析

- **收敛速度与最终 MSE 均随 α 单调改善**：α 从 0.01 增至 50，SGD 的最终 MSE 从 0.00895 降至 0.000001（约 9000 倍），Batch 从 0.04496 降至 0.000005（约 9000 倍）；在对数坐标下，曲线族清晰展现了这一单调规律；
- **SGD 对小学习率的容忍度更高**：$\alpha = 0.01$ 时 Batch 的最终 MSE（0.045）是 SGD（0.009）的 5 倍，因为 Batch 每 epoch 只更新 1 次，对小 α 更为敏感；
- **大学习率（$\alpha \geq 5$）未出现发散**：Sigmoid 导数 $\sigma'(v) = y(1-y) \leq 0.25$ 使单步更新上限为 $\alpha/4$；当权重增大后 $y(1-y) \to 0$ 进一步收缩梯度，系统自我稳定。这一特性是有代价的：权重较大时梯度趋零（梯度消失），这正是深层网络改用 ReLU 的动机。

6. 不同初始化权重范围对结果的影响

(1) 单种子结果（seed=42）

固定 $\alpha = 0.1$，测试初始化范围 $[-0.1, 0.1]$、$[-0.5, 0.5]$、$[-1.0, 1.0]$、$[-2.0, 2.0]$：

```
不同权重初始化范围的最终 MSE (10000 epochs, alpha=0.1, seed=42)
     Init Range |    SGD MSE |  Batch MSE
-----------------------------------------------
     [-0.1,0.1] |   0.000698 |   0.003136
     [-0.5,0.5] |   0.000699 |   0.003145
     [-1.0,1.0] |   0.000699 |   0.003158
     [-2.0,2.0] |   0.000701 |   0.003198
```

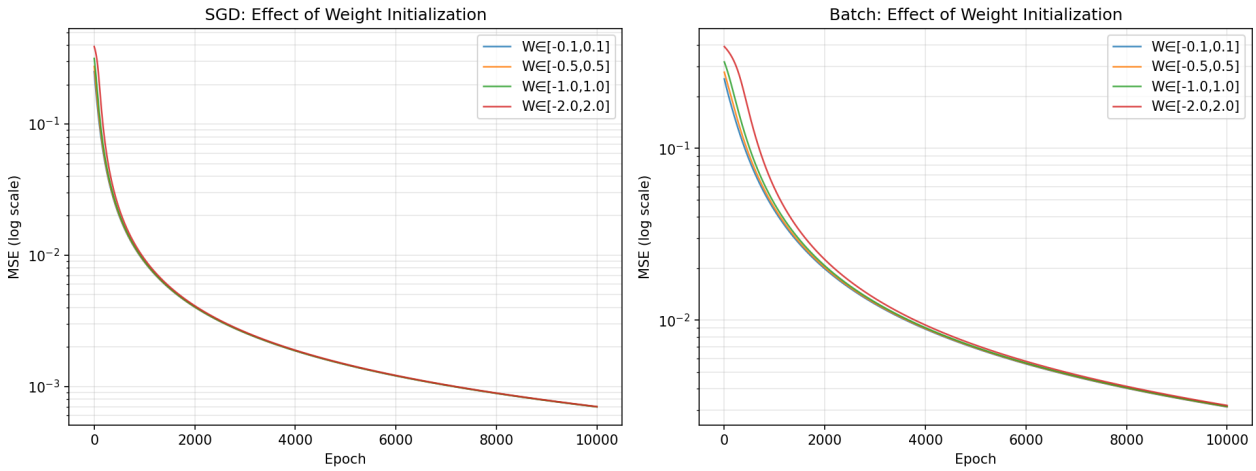


Figure 4: 不同权重初始化范围下 SGD（左）与 Batch（右）训练的 MSE 收敛曲线（对数坐标）

(2)　多随机种子鲁棒性验证（20 seeds）

为排除单一种子的偶然性，对每种初始化范围各运行 20 个随机种子（seed=0 至 19），统计最终 MSE 的均值与标准差：

```
多随机种子结果 (10000 epochs, alpha=0.1, 20 seeds)
  Init Range |    SGD mean±std    |    Batch mean±std
------------------------------------------------------------
  [-0.1,0.1] | 0.000698 ± 0.000000 | 0.003134 ± 0.000003
  [-0.5,0.5] | 0.000698 ± 0.000001 | 0.003135 ± 0.000015
  [-1.0,1.0] | 0.000698 ± 0.000002 | 0.003135 ± 0.000030
  [-2.0,2.0] | 0.000698 ± 0.000004 | 0.003129 ± 0.000075
```

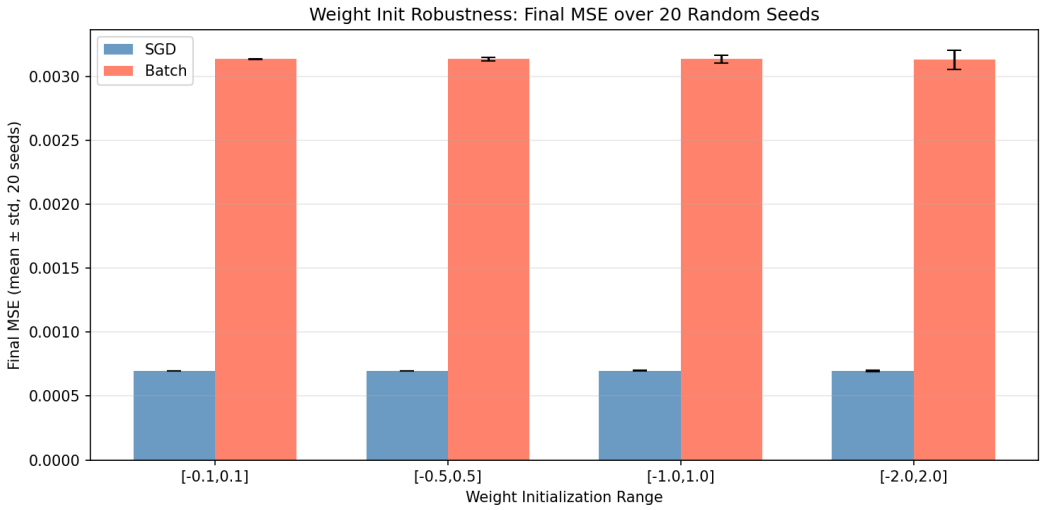


Figure 5: 20 个随机种子下不同初始化范围的最终 MSE 均值与标准差（误差棒）

综合分析

- **SGD 极度稳定**：跨所有初始化范围，20 个种子的 MSE 标准差不超过 4×10^{-6}，凭借频繁更新能从各种初始状态有效收敛；
- **Batch 轻微敏感**：标准差随初始化范围扩大从 3×10^{-6} 增至 7.5×10^{-5}；初始化越大，Sigmoid 更易饱和，梯度 $y(1-y)$ 更小，收敛略慢，种子间差异也略大；
- **初始化影响随 epoch 增大而递减**：图 4 中不同初始化范围的曲线在早期有所分散（大范围初始化使 Sigmoid 更易饱和、梯度更小，收敛起步更慢），但随训练推进逐渐汇聚，最终 MSE 差异可忽略不计，说明初始化范围的影响是暂时性的而非持续性的；
- **整体鲁棒性良好**：20 个种子中均未出现慢收敛或局部最优异常，验证了 $d = x_1$ 任务损失曲面的良好凸性；
- 在更深的网络中，初始化范围对训练稳定性影响显著（梯度爆炸/消失），实际工程通常采用 Xavier 或 He 初始化方案。

7. 实验总结

本次实验手动实现了基于 Sigmoid 激活函数的单层感知器，以 $d = x_1$ 映射为训练任务（含偏置项），分别通过 SGD 和 Batch 方法结合 Delta 规则完成了神经网络训练，并系统分析了学习率和权重初始化范围的影响。

主要结论

(1) **SGD vs Batch**：在相同 epoch 数和学习率下，SGD 的收敛速度和最终精度均优于 Batch（SGD MSE 约为 Batch 的 1/4.5），根本原因是 SGD 每个 epoch 执行 N 次权重更新而 Batch 只执行 1 次。若以总更新次数（而非 epoch 数）作为横轴进行公平比较，两者收敛曲线将趋于一致；

(2) **学习率的影响**：α 越大，收敛越快，10000 轮后最终 MSE 越低。本简单任务中大学习率（$\alpha = 1.0$）仍能稳定收敛；实际复杂网络中需权衡收敛速度与训练稳定性；

(3) **权重初始化的影响**：对 SGD 几乎无影响；对 Batch 有轻微影响——初始权重绝对值越大，收敛略慢。这是由于大初始权重导致 Sigmoid 饱和、梯度减小所致。在多层网络中，初始化方案（Xavier/He）的选取会对训练产生决定性影响。

装 订 线

二、 附加实验

1. 附加——Mini-Batch 实现与三方对比

```python
# MiniBatch.py
import numpy as np
from Sigmoid import Sigmoid

def DeltaMiniBatch(W, X, D, alpha=0.1, batch_size=2):
    N = X.shape[0]
    idx = np.random.permutation(N)          # 每 epoch 随机打乱
    for start in range(0, N, batch_size):
        batch_idx = idx[start:start + batch_size]
        m = len(batch_idx)
        dW = np.zeros_like(W)
        for k in batch_idx:
            x = X[k, :].T
            d = D[k]
            v = (W @ x)[0]
            y = Sigmoid(v)
            e = d - y
            dW[0][0] += alpha * e * y * (1 - y) * x[0]
            dW[0][1] += alpha * e * y * (1 - y) * x[1]
            dW[0][2] += alpha * e * y * (1 - y) * x[2]
        W = W + dW / m                      # mini-batch 内取平均
    return W
```

Listing 5: Mini-Batch 训练函数（每个 mini-batch 内取平均后更新，每 epoch 共 N/bs 次更新）

Mini-Batch 实现

三方对比结果

```
附加 Part 7: Mini-Batch (batch_size=2) 与 SGD/Batch 对比
 Mini-Batch 最终权重: [ 6.49241225 -0.22676497 -3.02370338]
   x1=0, x2=0, 期望:0, 输出:0.0464
   x1=0, x2=1, 期望:0, 输出:0.0373
   x1=1, x2=0, 期望:1, 输出:0.9698
   x1=1, x2=1, 期望:1, 输出:0.9624
 Mini-Batch 最终 MSE: 0.001467

    Method |  Final MSE | Updates/epoch
-----------------------------------------
       SGD |    0.000699 |        N=4
Mini-Batch |    0.001467 |     N/bs=2
     Batch |    0.003145 |          1
```

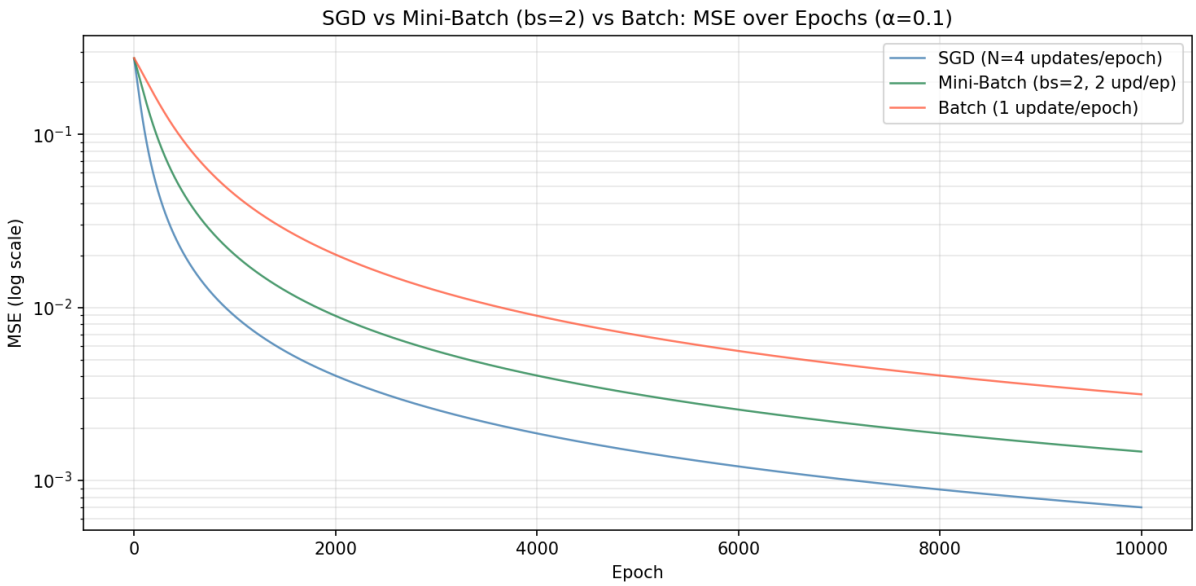


Figure 6: SGD、Mini-Batch（bs=2）与 Batch 三种方法的 MSE 收敛曲线对比（对数坐标，$\alpha = 0.1$）

分析

- **三方排序符合理论**：最终 MSE 严格满足 SGD(0.000699) < Mini-Batch(0.001467) < Batch(0.003145)，精确对应每 epoch 更新次数之比 4 : 2 : 1，验证了”每 epoch 更新次数越多，同等 epoch 数下收敛越好”的规律；
- **Mini-Batch 的折中价值**：Mini-Batch 在更新频率和梯度估计方差之间取得平衡——相比 SGD 每次只用 1 个样本（高方差），Mini-Batch 用 2 个样本的平均梯度更新（方差减半），在实际大规模数据集上可以充分利用硬件并行性，通常是工程实践中的首选；
- **推广意义**：对于 $N = 4$ 的微型数据集，Mini-Batch 优势不明显；在 $N = 10^4$ 至 10^6 的实际任务中，batch_size 通常取 32–512，在收敛速度、内存占用和梯度质量之间找到最佳权衡点。

装 订 线